Lecture Two

Study handout — text extracted from the original slides (wording preserved). Logic symbols fixed (∧ ∨ ¬ ⇒ ⇐ ≡ ≠ ⊥). About half the paper or less. 78 slides. Black text on white.

How to use: skim slide titles, study bullets in order, read formulas as written.

Slide 1 — Department of Computer Sciences

(No body text on this slide.)

Slide 2

أهداف الكلية رسالة الكلية الكلية
رؤية
عملعداد خريج متميز قادر على المنافسة في سوق الإ
ه 1.
اعتماد طرق تدريس متميزة وبرامج تعليمي
احليمجالها مي ن تصبح رائدة فأب ة
تلتزم الكلي
والقدرة على العمل مع وعالية الجودة وتتماشى مع متطلباتة
شامل
ا من خالل تقديمها لبرامج أكاديمية
وعالمي
.
االخرين من التخصصات االخرى
المجتمع وسوق العمل
رتوجيه االبحاث التطبيقية لتخدم أغراض رؤية مص
2. من خالل خلق بيئة
وإجراء البحوث اإلبداعية
متميزة في الدراسات العلياة.
ومهني
.
0302 تمكن من االستغالل
إدارية وأكاديمية محفزة
تي
استمرار الكلية في تطوير وإدارة البرامج ال
بناء شراكة مجتمعية فاعلة مع القطاعين العام
3. مثل للموارد والتكنولوجيا الحديثة
أل .
والخاص والدولي
بناء الشراكة مع المجتمع المحلي
تخدم احتياجات سوق العمل والباحثين
تحقيق تميز في أألداء االألكاديمي واإلداري والمهني.
ن 4.
طرح دورات تدريبية وبرامج مهنيه للعاملي
.
والمتخصصين
التوسع في تطبيق معايير الجودة الشاملة.
.5
في المنظمات المختلفة
استحداث برامج دراسية تتماشى مع رؤية مصر
6. اإلدارية كجزء من نشاط
تدريب الكوادرجيين
التحسين المستمر من خالل تثقيف الخري
.
2030
خدمات المجتمع وتعزيز الفضول الفكريمع
وترسيخ مبادئ التعلم مدى الحياة لديهم
تطوير الخدمات المجتمعية والبيئية.
.7
والمرونة للنمو اجات
توفير البنية التحتية التي توفر احتي
توبالتوسع في قبول الطالب الوافدين وتحسين مس

.8 التكنولوجيا واألساليب الجديدة في
تطوير
الطالب وتحديثها بما يضمن تميزها وتميز
.
الخدمة المقدمة لهم
اإلحصاء وعلوم الحاسب ونظم وتكنولوجيا ا
خريجيه
تحسين البنيـة التحتية للكلية.
.9
المعلومات
سينتنمية المهارات اإلبداعية والتفكير النقدي للدار.
.10

Slide 3

Faculty of Graduate Studies for Statistical Research Department of Computer Sciences Course title: Advanced Topic in Software Engineering Course Code: CS 601 Dr. Fatma A. El -Licy

Slide 4 — Lecture Two

(No body text on this slide.)

Slide 5 — S Exp Class

HERE IS
HOW ⊥
⊥ Incomplete OK E xp = F(S)
T T ⊥
T Inconsistent T ⊥
⊥ ⊥ ⊥
T ⊥ ⊥
T
T OK T T
16 October 2026 5

Slide 6 — Theorem Proving: Proof Rules

• Finding out if a binary expression is a theorem or antitheorem is called
PROVING.
Five rules for proving 1. Axiom Rule: If a binary expression is an axiom, then it is a
theorem. If a binary expression is an Anti-axiom, then it is an anti-theorem.
2. Evaluation Rule: If all the binary subexpressions of a binary expression are classified, then it is classified according to the truth tables.
16 October 2026 6

Slide 7

Theorem Proving: Proof Rules 3. Completion Rule: If a binary expression contains unclassified binary subexpressions, and all ways of classifying them place it in the same class, then it (expression) is in that class.
4. Consistency Rule: If a classified binary expression contains unclassified binary subexpressions, and only one way of classifying them is consistent, then they are classified that way.
5. Instance Rule: If a binary expression is classified, then all its instances have that same classification.
16 October 2026 7

Slide 8 — First Proof Rule: Axiom

• An axiom is a binary expression that is stated to be a theorem.
• An anti-axiom is similarly a binary expression stated to be an antitheorem.
• the grass is green, Axiom
• the sky is green, an Anti-axiom
• there's life elsewhere in the universe, Unclassified
• intelligent messages are coming from space, Unclassified
• intelligent messages are coming from space ⇒ there is life elsewhere Axiom 16 October 2026 8

Slide 9 — First Proof Rule: Axiom

• Axiom: ⊤
Theorem
• Anti-axiom: ⊥
Antitheorem
• Axiom: grass is green
Theorem
• Anti-axiom: sky is green Antitheorem
• Axiom: (intelligent messages are coming from space)
⇒ (there is a life elsewhere in the universe)
Theorem 16 October 2026 9

Slide 10

Second Proof Rule: The Evaluation If all the subexpressions are classified, then use the truth tables.
• If you know what the operands are, then the truth tables tell you what the whole expression is.
16 October 2026 10

Slide 11 — Third Proof Rule: The Completion

• If an expression contains unclassified subexpressions, and all ways of classifying them place it (expression) in the same class, then it is in that class.
• If you don't know what the operands are, you might still be able to tell what the whole expression is.
• Example: Theorem: there is life elsewhere in the universe ∨ T.
if there's life elsewhere, then that’s T ∨ T ≡ T.
If there isn't life elsewhere, then that's ⊥ ∨ T ≡ T.
It's a theorem by completion rule

Slide 12 — Third Proof Rule: The Completion Examples:

Theorem: (a) ∨ ¬ (a) because T ∨ ⊥ = ⊥ ∨ T = T
• Antitheorem: p = (a) ∧ ¬ (a)
Assume a= T then p = T ∧ ⊥ = ⊥
Assume a= ⊥ then p = ⊥ ∧ T = ⊥
16 October 2026 12

Slide 13 — Fourth Proof Rule: The Consistency

• If a classified binary expression contains binary subexpressions, and only one way of classifying them is consistent, then they are classified that way.
• This time, we know what the whole expression is, and we're wondering
what the parts might be.
Example 1: x, x ⇒ y are theorems, what is y?
Assume y is an antitheorem,
x ⇒ ⊥ ≡ T ⇒ ⊥ ≡
¬ T

∨⊥ ≡ ⊥ ∨ ⊥ ≡ ⊥
i.e., x ⇒ y is antitheorem contradicting the premises therefore, y is not antitheorem, i.e., y is a theorem 16 October 2026 13

Slide 14 — Example 2:

Theorem: ¬ x, What's x?
• Assume x = T i.e., assume x is a theorem
• Apply the evaluation rule, ¬ x = ⊥ is an antitheorem, contradicting that (¬ x) is a theorem.
• So x has to be an antitheorem.
• This example means no need to talk about Antiaxioms or antitheorems.
• If we want to say something is an antitheorem, we just say its negation is a theorem.
• (We also don't need to have both true and false)

Slide 15

Fifth Proof Rule: The Instance If a binary expression is classified as a theorem or an antitheorem, then all its instances have that same classification.
axiom: x = x. Theorem: x = x
Theorem: (T= ⊥∨⊥) ≡ (T= ⊥∨⊥)
16 October 2026 15

Slide 16

Second Proof Rule: The Evaluation Exp ≡ F(value1, Value2,….Value n)
≡ F(classified subexp) then just compute it Third Proof Rule: The Completion Exp ≡ F(set of classified and unclassified subexp)
then use truth table Fourth Proof Rule: The Consistency Classified Exp ≡ F(set of classified and unclassified subexp)
then assume values and apply the consistency rule 16 October 2026 16
UNIVERSITY

Slide 17 — Classification of Logical Systems

• Classical logic: all five rules, (which we are using in this course)
• Constructive logic: don't allow the completion. In a logic course, it's interesting to see what you can do if we DO NOT allow completion rule. (do not try several states, one may only substitute the known values of the subexp., and/or apply consistency rule)
• Evaluation logic: don't allow the consistency rule and the completion rule.
16 October 2026 17

Slide 18 — EXPRESSION and PROOF FORMAT

• x∧ y ∨ z leave more space around operators with less precedence x ∧ y∨ z is misleading, and causes errors.
• When an expression is too long to fit on one line, it should be broken at a sensible place, which usually means at the main connective.
(First part)
∧ (Second part)
• Put left brackets on the left, and right brackets on the right.
• Here's a continuing equation, nicely formatted.
Expression 0 Expression 0 = Expression 1 means
= Expression 1 ∧ Expression 1 = Expression 2
= Expression 2 ∧ Expression 2 = Expression 3
= Expression 3 16 October 2026 18

Slide 19 — Expression and Proof Format

• Do Do Not
• x∧ y ∨ z x ∧ y∨ z (First (First part)
part) ∧ (Second ∧ (Second part)
part)
Expression 0 Hint 0 (tells why expression 0 equal expression 1
= Expression 1 Hint1
= Expression 2 Hint2
= Expression 3 16 October 2026 19

Slide 20 — Expression and Proof Format: Example

• Example: Prove: a∧ b ⇒ c ≡ a ⇒ (b ⇒ c)
§ a ∧ b ⇒ c Material implication ≡
¬ (a∧ b)
∨ c Duality ≡ ¬ a ∨ ¬ b ∨ c Material implication ≡ a ⇒ ¬ b ∨ c Material implication ≡ a ⇒ (b ⇒ c)
Material implication: x ⇒ y ≡ ¬ x ∨ y a ∧ b ⇒ c =
¬ (a ∧ b)
∨ c 16 October 2026 20

Slide 21 — Proof of Expression Format: Example

• Simplify to true: a ∧ b ⇒ c ≡ a ⇒ (b ⇒ c)
§ a ∧ b ⇒ c ≡ a ⇒ (b ⇒ c) Material implication 3 times = (
¬ (a ∧ b)
∨ c ≡ ¬ a ∨ (¬ b ∨ c)) Duality = (
¬ a ∨¬ b ∨ c ≡ ¬ a ∨ ¬ b ∨ c) Reflexivity of ≡
= T 16 October 2026 21

Slide 22

Read The Handout Carefully, To Realize The Precedence and The Rules.
You Should Know The Logic and Numerals Laws.
Memorize As Much, or at Least Know Where To Find Them Solving the Assignments are essential for Training understanding and passing the course 16 October 2026

Slide 23 — NOW YOU CAN FINISH ASSIGNMENT ONE 16 Oct…

UNIVERSITY

Slide 24 — Before completing the theory of binary, …

• What is the inverse and converse of a conditional statement?
• What is the relation between them (the inverse and converse of a conditional statement) and the original statement?
16 October 2026 24

Slide 25 — Logical Ordering Operator (⇒) Expressions

VS its Inverse, Converse
A logical expression that has a preceding and consequent (e.g. a ⇒ b) is neither, logically equivalent to its inverse nor it is equivalent to its converse.
{Inverse of a logical statement (A ⇒ B)} IS (~A ⇒ ~B)
Converse of a logical statement (A ⇒ B)} IS (B ⇒ A)
16 October 2026 25

Slide 26 — Logical Ordering Operator (⇒) Expression…

A logical expression that has a preceding and consequent (e.g. a ⇒ b)
is not logically equivalent to its inverse.
{Inverse of a logical statement (A ⇒ B)} IS (~A ⇒ ~B)
(A ⇒ B) ≠ (~A ⇒ ~B)
Proof LHS: (A ⇒ B) ⇔ ~A ∨ B RHS: (~A ⇒ ~B) ⇔ A ∨ ~ B Notice that (A ⇒ B) = (~B ⇒ ~A) Contraposition rule

Slide 27 — Logical Ordering Operator (⇒) Expression…

A logical expression that has a preceding and consequent (e.g. a ⇒ b)
is not logically equivalent to its converse.
Converse of a logical statement (A ⇒ B)} IS (B ⇒ A)
(A ⇒ B)} ≠ (B ⇒ A)
Proof:
LHS: (A ⇒ B) ⇔ ~A ∨ B RHS: (B ⇒ A) ⇔ A ∨ ~ B 16 October 2026 27

Slide 28 — Binary Theory Continued

(No body text on this slide.)

Slide 29 — Monotonicity & Context

ARE Advanced Tricks of Binary Theory For Solving Problems Fast 16 October 2026 29

Slide 30 — Monotonicity and Anti-monotonicity

• Monotonicity and Anti-monotonicity.
• Covariance and Contravariance.
• Varies directly as and Varies inversely as.
• Nondecreasing and Nonincreasing
• Sorted and Sorted backwards.
16 October 2026 30

Slide 31 — Monotonic And Anti -monotonic

• Number:
x ≤ y x is less than or equals to y ∞ ≤ +
∞ 0 ≤ 1 smaller ≤ larger x ≤ y ⇒ f x ≤ f y f is monotonic as x getting larger f x getting larger (or equal)
x ≤ y ⇒ f x ≥ f y f is anti-monotonic as x getting larger f x getting smaller (or equal)
16 October 2026 31

Slide 32 — Monotonic and Anti -monotonic

• Binary:
x ⇒ y x is stronger than or equals to y ⊥ ⇒ T stronger ⇒ weaker (x ⇒ y)
⇒ (f x ⇒ f y) f is monotonic as x getting weaker f x getting weaker (or equal)
(x ⇒ y)
⇒ (f x ⇐ f y) f is antimonotonic as x getting weaker f x getting stronger (or equal)
16 October 2026 32

Slide 33

Monotonic and Anti-monotonic Rules ¬ x antimonotonic in x x ∧ y monotonic in x monotonic in y x ∨ y monotonic in x monotonic in y x ⇒ y antimonotonic in x monotonic in y x ⇐ y antimonotonic in y monotonic in x
If a then b else c monotonic in b monotonic in c (a ⇒ b) ∧ (¬ a ⇒ c)
≡ (a ∧ b) ∨ (¬ a ∧ c)
16 October 2026 33

Slide 34 — Monotonic and Anti-monotonic

Example a ⇒ a ∨ b Law of generality ¬ (a ∧ ¬ (a∨ b)) use law of generality to replace a for (a ∨ b)
a ⇒ a ∨ b ≡ a ∨ b ⇐ a monotonic in a ∨ b ⇐ ¬ (a ∧ ¬ a) law of noncontradiction = T Other than ¬ (a ∧ ¬ (a∨ b)) = ¬ a ∨ (a∨ b) = (¬ a ∨ a)∨ b = T ∨ b = T 16 October 2026 34

Slide 35 — Context

• In a ∧ B, a is subexpression in B, then when changing B we can assume the subexpression a (to be an axiom)
Example: see next slide
• In A ∧ b b is subexpression in A, when changing A we can assume the subexpression b (to be an axiom)
16 October 2026 35

Slide 36 — Context

• In x ∧ Y, x is subexpression in Y, then when changing Y, we can assume the subexpression x (to be an axiom)
EXAMPLE:
x Y ¬ (a ∧ ¬ (a ∨ b)) assume a in the expression (a ∨ b)
= ¬ (a ∧ ¬ (T ∨ b)) base law for ∨ (symmetry law)
= ¬ (a ∧ ¬ T) Truth Table for ¬
= ¬ (a ∧ ⊥) base law for ∧
= ¬ ⊥ binary axiom (Truth Table for ¬)
= T 16 October 2026

Slide 37

• In both a ∧ B, B ∧ a when changing B we can assume a
• In both a ∨ B, B ∨ a when changing B we can assume ¬ a
• In a ⇒ B, when changing B we can assume a Context
• In A ⇒ b, when changing A we can assume ¬ b Rules
• In if A then b else c fi, when changing A we can assume b ≠ c
• In if a then B else c fi, when changing B we can assume a
• In if a then b else C fi, when changing C we can assume ¬ a
θ small letter is a subexpression in The capital letter’s expression 16 October 2026 37

Slide 38 — FORMALISM

• One use computers to solve problems, or to provide services, or just for fun.
• The desired computer behavior is usually described:
¬ At first; informally, in a natural language (like English), ¬ Then, may be clarified through diagrams, perhaps with some hand gestures, rather than formally ¬ Eventually: The desired computer behavior is described formally as a program.
16 October 2026 38

Slide 39 — FORMALISM

• A PROGRAMMER must be able to translate informal descriptions to formal descriptions.
• Astatement in a natural language can be vague, ambiguous, or subtle, and can rely on a great deal of cultural context and application area.
• This makes formalization difficult, but also necessary.
• We cannot possibly say how to formalize, in general; it requires a thorough knowledge of the natural language.
16 October 2026 39

Slide 40

FORMALISM Some pitfalls in the translation from English to binary expressions.
• The words “and ”, “also”, “but”, “yet”, “however ”, although, and
“moreover ”
might all be translated as ∧. Just putting things next to each other sometimes means ∧.
“They're red, ripe, and juicy, but not sweet.”≡ red ∧ ripe ∧ juicy∧ ¬sweet
• The word “or” translated as ∨, and sometimes as □ .
“They're either small or bitter” ≡ small ∨ bitter. But
“Either we eat them, or we preserve them” ≡ eat □ preserve.
16 October 2026 40

Slide 41

FORMALISM Some pitfalls in the translation from English to binary expressions.
• The word “if” is sometimes best translated as ⇒
and sometimes as =
“If it rains, we'll stay home.” probably leaves open the possibility that we might stay home even if it doesn't rain, and should be translated as rain ⇒ home.
But “If it snows, we can go skiing.” probably also means “and if it doesn't, we can't”, and is best translated as snow=ski.
16 October 2026 41

Slide 42 — DO ASSIGNMENT TWO 16 October 2026 42

(No body text on this slide.)

Slide 43 — Number Theory 16 October 2026

(No body text on this slide.)

Slide 44 — Number Theory

• In this course the numbers expression is formed in the following ways.
• Asequence of one or more decimal digits, optionally including a decimal point, with the following operations
∞
“infinity”
x / y
“ x divided by y ”
y – x
“minus x ”
x
“x to the power y”
x + y
“ x plus y ”
x ↑ y
“ x max y ”
x – y
“ x minus y ”
x ↓ y
“ x min y ”
x × y
“ x times y ”
If a then x else y fi where x and y are any number expressions, and a is any binary expression.
16 October 2026 44

Slide 45 — Number Theory

• numbers are joined by relational operators (<, ≤, >, ≥, =, ≠) to make Binary Expressions x < y “ x is less than y ”

x ≤ y “ x is less than or equal to y ”
x > y “ x is greater than y ”
x ≥ y “ x is greater than or equal to y ”
x = y “ x equals y ”, “ x is equal to y ”
x y “ x differs from y ”, “ x is unequal to y ”
16 October 2026

Slide 46

NUMBER THEORY AND DOMAIN Domain_addendum -lecture2.pptx 16 October 2026 46

Slide 47 — The Axioms of Number Theory

• are listed in the handout. They should be familiar to you already.
• Notice particularly the two axioms:
– ∞ ≤ x ≤ ∞ extremes – ∞ < x ⇒ ∞ + x = ∞ absorption
• Number Theory is incomplete. For example, the binary 1/2 expressions 1/0 = 5 and 0 < (–1) = i (complex number)
can neither be proven nor be disproven.
(unprovable)
16 October 2026 47

Slide 48 — Character Theory 16 October 2026

(No body text on this slide.)

Slide 49 — Character Theory

• The simplest character expressions are written as a graphical shape enclosed by left and right double-quotes.
“A ” is the “capital A” character, “1” is the “one ” character, and
“ ” is the “space” character.
“““” left double-quote
“””” right double-quote
• Character Theory is trivial. It has operators: succ (successor), pred (predecessor), and ↑ ↓ = < ≤ > ≥ if then else fi.
16 October 2026 49

Slide 50 — General Laws 16 October 2026

(No body text on this slide.)

Slide 51 — if then else fi,

• All our theories use the operators =
• Their laws are listed in the handout under the heading
“Generic”, meaning that they are part of every theory.
• The operators ↑ ↓ < ≤ > ≥
 Apply to some, but not all, types of expression.
 Whenever they do apply, their axioms, are listed under the heading “Generic” in the handout, go with them.
16 October 2026 51

Slide 52 — DO ALL ASSIGNMENT TWO 16 October 2026 52

(No body text on this slide.)

Slide 53 — Data Structure 16 October 2026

(No body text on this slide.)

Slide 54 — Data Structure

• Bunch unpackaged unindexed
• Set packaged unindexed
• String unpackaged indexed
• List packaged indexed 16 October 2026 54

Slide 55 — Bunch Theory 16 October 2026

(No body text on this slide.)

Slide 56 — BUNCH THEORY

• Abunch is a mathematical data structure used to represent a collection of things.
• Any number, character, or binary value (and later also set, string of elements, and list of elements) is an elementary bunch, or element.
0, 5, 3, 9 and A, 7, &, ⊥ are bunches
• Given the bunches A and B
A, B “ A union B ”
A ‘ B “ A intersection B ”
16 October 2026 56

Slide 57 — Bunch Theory

• Abunch is a mathematical data structure used to represent a collection of things.
¢A is “size of A ”, “cardinality of A”
¢2 = 1 ¢(0, 2, 5, 9, 9) = 4
• The Binary inclusion:
A:B “ A is in B” or “ A is included in B”
2: 0, 2, 5, 9 2: 2 9: 0, 5, 3, 9
A bunch consists of its elements (not contains its elements)
16 October 2026 57

Slide 58

Bunch Theory: AXIOMS Let x and y be two elements (elementary bunches), and
A, B, and C be arbitrary bunches, then x: y ≡ x=y elementary axiom x: A, B ≡ x: A ∨ x: B compound axiom
A, A ≡ A A ‘A ≡ A idempotence
A, B = B, A A ‘B ≡ B ’A symmetry
A, (B, C) ≡ (A, B), C A ‘(B ‘C) ≡ (A ‘B) ‘C associativity
A, B: C ≡ A: C ∧ B: C anti-distributivity
A: B ‘C ≡ A: B ∧ A: C distributivity 16 October 2026 58

Slide 59 — Bunch Theory: AXIOMS

A: A, B generalization A ‘B: A specialization
A: A reflexivity A: B ∧ B: A ≡ A=B anti-symmetry
A: B ∧ B: C ⇒ A: C transitivity Colon (inclusion of the Bunch) is ordering (reflex + anti-symmetry + trans)
Colon is the ordering relation on Bunch (like ≤ in numbers and ⇒ in Binary)
Remember

¢x = 1, bunch size
¢ (A, B) + ¢(A‘B) = ¢A + ¢B size (bunch is not idempotent)
¬ x: A ≡ ¢(A ‘x) = 0 size A: B ⇒ ¢A ≤ ¢B size 16 October 2026 59

Slide 60

Bunch Theory: USEFUL BUNCHES null the empty bunch bin = ⊤, ⊥ the binary values nat = 0, 1, 2,... int =..., –2, –1, 0, 1, 2,...
1/2 rat =..., –1, 0, 2/3,... real =..., 2,... the real numbers xnat = 0, 1, 2,.. ∞ + 1 the extended natural numbers xint = –∞,.. –2, –1, 0, 1, 2,.. ∞ + 1 the extended integer numbers xrat = –∞,..., –1, 0, 2/3,.. ∞ + 1 the extended rational numbers xreal = –∞,.. ∞ + 1 the extended real numbers char =..., “a”, “A ”,..”Z ” the character values
16 October 2026

Slide 61 — Bunch Theory: NOTATIONS

• x,..y x ≤ y for integers up to infinity this is an axiom only for integer items/numerals
• i: x,..y ≡ x≤ i<y
• i:5,..19 ≡ 5 ≤ i< 19
• ¢(x,..y) ≡ y-x 0,..3 ≡ 0,1,2 0,..1 ≡ 0 0,..0 ≡ null 0,..∞ ≡ nat 0,..∞ +1 ≡ xnat 16 October 2026 61

Slide 62 — Bunch Theory: Distribution Property

• Most operators distribute over bunch union.
• -null ≡ null
• – (A, B) ≡ –A, –B -(1, 4, 6, 9) ≡ -1, -4, -6, -9
A+ null ≡ null ≡ null + A null + 10 = null (A, B) + (C, D) ≡ A+ C, A+D, B+C, B+D (1, 2) + (10, 20) = 11, 12, 21, 22 (1,2) + 10 = 11, 12 1+ 10 = 11 nat + 2 ≡ 2, 3, 4, 5, 6,… nat × 2 ≡ 0, 2, 4, 6,…
2 nat nat ≡ 0, 1, 4, 9, 16,… 2 = 1, 2, 4, 8, …
16 October 2026 62

Slide 63 — DATA STRUCTURE

• Bunch unpackaged unindexed
• Set packaged unindexed 16 October 2026 63

Slide 64 — set Theory 16 October 2026

(No body text on this slide.)

Slide 65 — SET THEORY

• S={bunch} S={A} set containing A
• ~S content of a set (bunch)
S ={1, 4, 8} is a set then ~S = 1, 4, 8 is a bunch
{1, 4, 8}, 9 a bunch of 2 elements
{{1, 4, 8}, 9} a set containing 2 elements a set and a number
• {null} the null set; {nat} the set of natural numbers
• {0,1,2} ≡ {0,..3}
• ${1, 4, 8} ≡ 3 size of the set (cardinality)
• ⊂ (0,1) = Power (0, 1) ≡ {null}, {0},{1},{0,1}

Slide 66 — SET THEORY

• Let S be a set, and let A and B be anything.
{A}
 A structure
• S={ 1,2,3,7} ≠ 1,2,3,7 yet ~S = 1,2,3,7 {~S}

≡ S set formation
• ~S = 1,2,3,7 then {~S }= {1,2,3,7} =S ~{A}
≡ A “contents”
${A} ≡ ¢A “size”, “cardinality”
• ${A }=${ 2, 3, 8, 9} = 4 = ¢A = ¢ (2,3,8,9) = 4 16 October 2026 66

Slide 67 — Set Theory

• Let S be a set, x be an element and let A and B be anything.
A ∈ {B} ≡ A: B “elements”; x ∈ {B} ≡ x: B
{A} □ {B} ≡ A: B “subset” {A}:⊂ B ≡ A: B “power ”
{A} □ {B} ≡ {A, B} “union”
{A} ∩ {B} ≡ {A ‘B} “intersection”
{A} = {B} ≡ A = B “equation”
16 October 2026 67

Slide 68 — DATA STRUCTURE

• Bunch unpackaged unindexed
• Set packaged unindexed
• String unpackaged indexed 16 October 2026 68

Slide 69 — String Theory 16 October 2026

(No body text on this slide.)

Slide 70 — STRING THEORY

• String are indexed sequences nil the empty string 8 a one-item string 4;5;8;9 a four-item string 4; 5; 8; 9 ↔ (4;5;8;9) = 4 string length 0 1 2 3 4 5 6 7 8 (4;5;8;9) = item in index 2 = 8
• At index n, the number of processed items (count) is n the next item to be processed is n 16 October 2026 70

Slide 71

String Theory (4;5;8;9) = 8;5;9 substring 2;1;3 Let n be an extended natural and S be a string, and i be an item
• n*S is a string composed of n copies of S joined together.
3 * (0; 1) ≡ 0; 1; 0; 1; 0; 1
• Without a left operand, *S means the bunch union of all strings formed by joining any number of copies of S.
*S ≡ nil, 1*S, 2*S, … (bunch union of strings)
*(0; 1) ≡ nil, 0; 1, 0;1;0;1,...
Now let n be an index of S and i be an item (not necessarily of S):
• S □ n□ i is a string like S except that the item at index n is i.
i.e., (store the item i in the index n)
(3; 5; 9)□ 1□ 8 ≡ 3; 8; 9

Slide 72 — String Theory

• 3; 6; 4; 7 < 3; 7; 2 (as if they were decimal numbers: 3647 < 3720)
• 3; 6; 4 < 3; 6; 4; 7 Lexicographic order.
• Let i be an item, S and T are strings, and n: xnat then are nil is the empty string, S;T is “ S join T ”
strings
S is “ item(s) in index T of S”, n*S is “ n copies of S ”
T
S □ n□ i is “ S but at n there's i ”,
*S “copies of S ” is a bunch of strings, (Kleene star after the logician Stephen Kleene), not a string ↔ S “length of S ” is an extended natural number.
16 October 2026 72

Slide 73

Axioms of String Theory Let S, T, and U be strings, i and j are items, and n is an extended natural number.
S; nil ≡ S ≡ nil; S identity
S; (T; U) ≡ (S; T); U associativity ↔ nil ≡ 0 base ↔ i ≡ 1 base ↔ (S; T) ≡ ↔ S + ↔ T idempotent
S ≡ nil ↔ S< ∞ ⇒ (S; i; T) ≡ i S ≡ S; S nil ↔ S T;U T U 16 October 2026 73

Slide 74

Application of String Axioms Let S =2; 3; 4; 5; 8; 9, T= 0; 2; 1, U = 4; 5 be strings and i = 10 is an item
• ↔ (S; T) ≡ ↔ S + ↔ T ↔ (S; T) ≡ ↔ (2; 3; 4; 5; 8; 9; 0; 2;1) = 9 ↔ S + ↔ T = 6 + 3 = 9
• ↔ S < ∞ ⇒ (S; i; T) ≡ i ↔ S 6 < ∞ ⇒ (2; 3; 4; 5; 8; 9; 10; 0; 2;1) ≡ 10
• S ≡ S; S S ≡ (2; 3; 4; 5; 8; 9) = 2; 4; 3; 8; 9
T;U T U T;U 0; 2; 1; 4; 5
S; S ≡ (2; 3; 4; 5; 8; 9); (2; 3; 4; 5; 8; 9) ≡ (2; 4; 3); (8; 9)
T U 0;2;1 4;5 = 2; 4; 3; 8; 9 ≡ S T;U 16 October 2026 74

Slide 75

Axioms of String Theory Let S, T, and U be strings, i and j are items, and n: xnat.
S = (S; S) □ = (S) 0*S = nil (T;U) T U □ T U □
(n+ 1)*S = n*S; S ↔S< ∞ ⇒ (S; i)□ ↔S □ j ≡ S; j↔S< ∞ ⇒ (S; i)□ ↔S □ j; T≡ S; j; T↔S< ∞ ⇒ nil ≤ S < S; i; T Order wise ↔S< ∞ ⇒ (i<j ⇒ S; i; T < S; j; U) Order wise ↔S< ∞ ⇒ (i=j ≡ S; i; T = S; j; T) Order wise 16 October 2026 75

Slide 76 — Axioms of String Theory

S, T and U be strings, i and j are items, and n:xnat.
□ (x;..y) x ≤ y for integers up to infinity, this is an axiom for only integer items/numerals
□ ↔ (x;..y)= y –x (string of integers starting at x up to but not included y)
□ ↔ (x;..x+1) = x+1–x =1 x;..x+1 ≡ x □ ↔ (x;..x) ≡ 0 x;..x ≡ nil
□ (x;..y); (y;..z) ≡ x;..z where x ≤ y ≤ z 16 October 2026 76

Slide 77 — Axioms of String Theory

θ “abcdefghij” = “def”
3;..6
θ String join distributes over bunch union:
let A= a, b, c B= r, s, t and D, E be bunch unions
• A; null; B ≡ null A; nil; B ≡ A; B
• (A, B); (D, E) why do we need parenthesis??
=A;D, A;E, B;D, B;E ≡ a;D, b;D, c;D, a;E, b;E, c;E, r;D, s;D,;D, r;E, s;E, t;E
θ YET A, B; D, E = A, B; D, E is bunch union of A and string B;D and E ≡ a, b, c, (r, s, t); D, E = a,b,c, r;D, s;D, t;D, E = A, r;D, s;D, t;D, E 16 October 2026 77

Slide 78 — TRY ASSIGNMENT THREE 16 October 2026 78

(No body text on this slide.)